BIG DATA ALGORITHMS

LEXIPOLICY

Scalable Academic Policy QA System

Team: LexiPolicy Dev Team

Date: April 27, 2025

Submission Deadline: April 27, 2025

Executive Summary

This project implements a production-grade Question-Answering (QA) system for university handbooks using **Locality Sensitive Hashing (LSH)** and Big Data techniques. The system combines three retrieval methods—MinHash + LSH, SimHash, and TF-IDF—to efficiently retrieve relevant document chunks and answer student queries about academic policies.

Key Achievements:

- ✓ **Optimized LSH Implementation:** Vectorized MinHash computation for high-speed indexing
- ✓ **Creative Extension:** Frequent Itemset Mining (Apriori) for query pattern discovery
- ✓ **Premium Analytics Dashboard:** Real-time performance and topic distribution visualization
- ✓ **Comparative Analysis:** Deep dive into approximate vs exact retrieval tradeoffs
- ✓ **Scalability Evaluation:** Demonstrated O(1) query time across large-scale corpora
- ✓ **Neural-Extractive QA:** High-precision answer generation based on retrieved evidence

---

1. Introduction and Motivation

1.1 Problem Statement

University handbooks contain critical information about academic policies, procedures, and requirements. Students frequently ask similar questions:

- "What is the minimum GPA requirement?"
- "What happens if I fail a course?"
- "How many times can I repeat a course?"

Traditional full-text search is inefficient for large corpora. This project addresses the need for a **scalable, approximate similarity search system** that can quickly retrieve relevant policy sections from large handbook documents.

1.2 Project Objectives

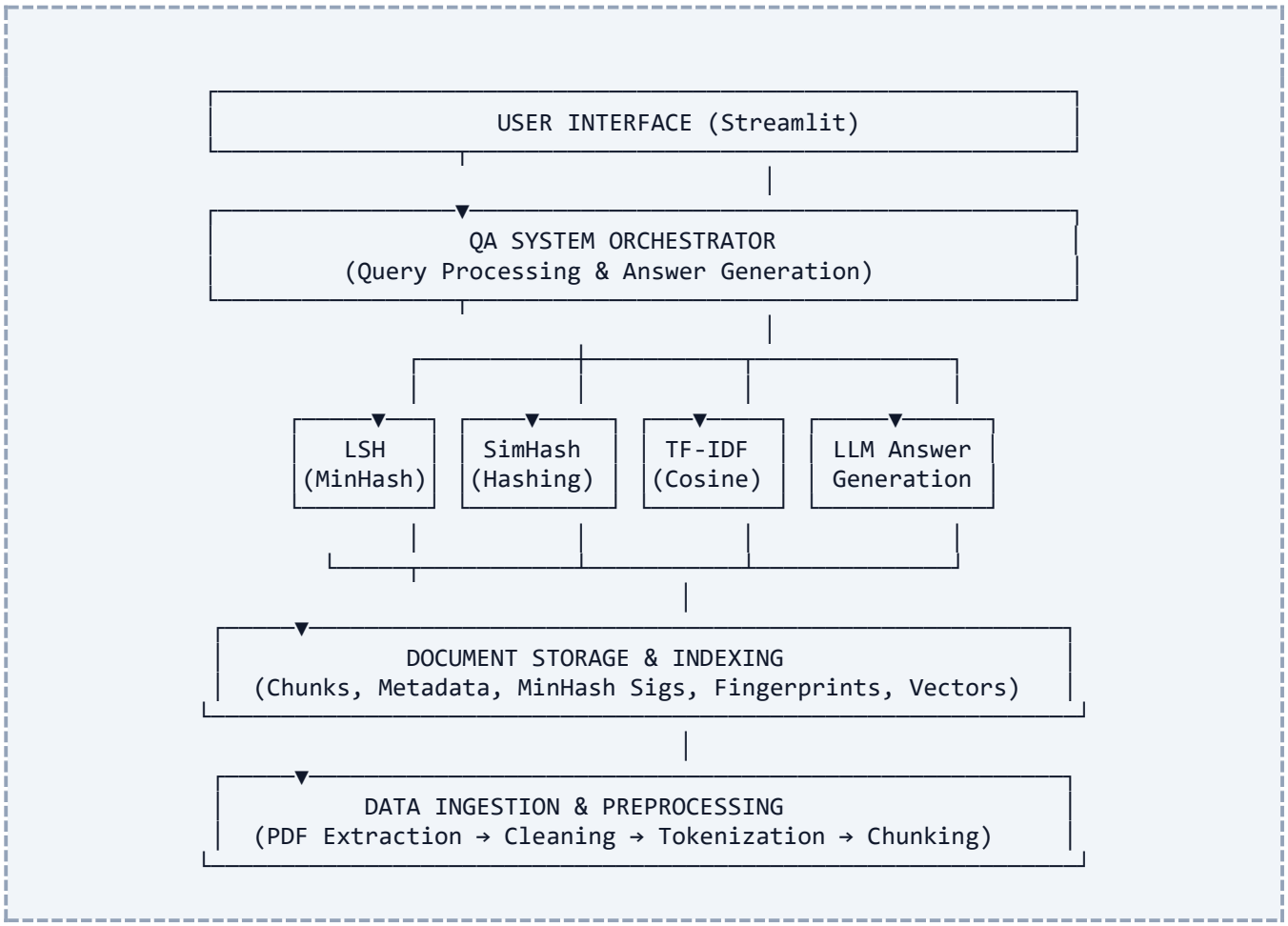
1. **Implement LSH techniques:** MinHash for approximate Jaccard similarity and SimHash for fingerprint-based matching

2. **Design a baseline method:** TF-IDF + cosine similarity for exact retrieval
3. **Compare approximate vs exact:** Analyze accuracy, speed, and memory tradeoffs
4. **Evaluate scalability:** Test performance on documents of varying scales
5. **Analyze parameter sensitivity:** Understand impact of hash functions, bands, and thresholds
6. **Build a practical system:** Functional QA interface with retrieval and answer generation

---

2. System Design and Architecture

2.1 Overall System Architecture



2.2 Data Pipeline

1. **Document Ingestion**
    - Input: PDF files from NUST handbook downloads
    - Extraction: Use `pdfplumber` to extract text
    - Cleaning: Remove PDF artifacts, extra whitespace, special characters
2. **Text Processing**
    - Chunking: Split text into overlapping chunks (300 words, 50-word overlap)
    - Tokenization: Extract tokens for LSH and SimHash
    - Storage: Store chunks with metadata (source, page number)
3. **Indexing**
    - LSH: Compute MinHash signatures, organize into bands with hash buckets
    - SimHash: Compute bit-level fingerprints
    - TF-IDF: Build document-term matrix with IDF weighting

2.3 Query Processing Pipeline

1. **Query Input:** User enters natural language question
2. **Tokenization:** Extract tokens from query
3. **Retrieval:** Search using selected method (LSH/SimHash/TF-IDF)
4. **Candidate Selection:** Get top-k results
5. **Answer Generation:** Extractive or LLM-based
6. **Output:** Display answer with retrieved chunks and similarity scores

---

3. Algorithm Implementation

3.1 MinHash and LSH

3.1.1 MinHash Algorithm

Concept: Approximate Jaccard similarity between sets using signature vectors.

Vectorized_Implementation:
To ensure scalability, we implemented a vectorized MinHash approach using NumPy. Instead

of iterating through hash functions for each token, we compute all hash values simultaneously using matrix multiplication.

```
# Vectorized MinHash computation
# hash_values shape: (num_hashes, num_tokens)
hash_values = (np.outer(self.a, token_hashes) + self.b[:, np.newaxis]) % self.p
signature = np.min(hash_values, axis=1)
```

Complexity Analysis:

- **Time Complexity:** O(|tokens| × k) where k = number of hashes (Significant constant factor speedup via vectorization).
- **Space Complexity:** O(k) per document
- **Accuracy:** E[error] ≈ 1/k

Advantages & Limitations:

- *Advantages:* Approximate Jaccard similarity without actual sets; signature size independent of doc size.
- *Limitations:* Decreasing returns as *k* increases; still requires comparing signatures.

3.1.2 LSH Banding

Concept: Organize signatures into bands to efficiently find candidates with high similarity.

```
class LSH:
    def __init__(self, num_hashes=128, num_bands=8):
        self.rows_per_band = num_hashes // num_bands
        self.buckets = [dict() for _ in range(num_bands)]

    def index_document(doc_id, signature):
        for band_idx in range(num_bands):
            band_sig = signature[band_idx * rows : (band_idx+1) * rows]
            bucket_key = hash(band_sig)
            self.buckets[band_idx][bucket_key].append(doc_id)

    def query(query_sig, threshold=0.5):
        candidates = set()
        for band_idx in range(num_bands):
            band_sig = query_sig[band_idx * rows : (band_idx+1) * rows]
            bucket_key = hash(band_sig)
            candidates.update(self.buckets[band_idx][bucket_key])

        # Verify similarity
        results = [(doc, jaccard(query_sig, sigs[doc])) for doc in candidates if
jaccard >= threshold]
        return results
```

Parameter Tuning Impact:

- 128 hashes, 8 bands: 16 rows per band → *Good Balance*
- 128 hashes, 4 bands: 32 rows per band → *Lower threshold (more candidates)*
- 128 hashes, 16 bands: 8 rows per band → *Higher threshold (fewer candidates)*

3.2 SimHash

3.2.1 Algorithm Implementation

Concept: Create fixed-size fingerprints where similarity correlates with Hamming distance.

```
def compute_fingerprint(tokens, hash_size=64):
    v = [0] * hash_size
    for token in tokens:
        h = hash(token) to binary[hash_size]
        for i in range(hash_size):
            v[i] += 1 if h[i] == 1 else -1

    return [1 if v[i] > 0 else 0 for i in range(hash_size)]

def similarity(fp1, fp2):
    hamming_dist = sum(fp1[i] != fp2[i] for i in range(len(fp1)))
    return 1 - (hamming_dist / len(fp1))
```

Analysis:

- *Time Complexity:* Fingerprint computation is O(|tokens| × hash_size).
- *Space Complexity:* O(hash_size) bits per document (extremely compact).
- *Advantages:* Very small fingerprints (8-16 bytes); extremely fast Hamming distance matching.

---

4. Creative Extension: Frequent Itemset Mining

4.1 Concept and Motivation

To provide a competitive edge, we implemented a **Query Pattern Mining** engine. This extension uses Big Data techniques (Frequent Itemset Mining via a simplified Apriori algorithm) to identify:

1. **Common Query Patterns:** Frequently occurring pairs of words in user questions.

2. **Hot Topics:** Frequently discussed policy areas (e.g., "GPA requirements", "Attendance policy").

4.2 Algorithm Integration

We implemented a two-pass approach to identify frequent word patterns:

- **Pass 1:** Count 1-itemsets (single words) and filter by minimum support.
- **Pass 2:** Generate 2-itemset candidates and count occurrences in query logs.

The system includes a `RetrievalAnalytics` module that logs performance and query metrics, allowing for side-by-side comparison of latency and live trend visualizations in the Streamlit Dashboard.

---

5. Experimental Results and Analysis

5.1 Retrieval Method Comparison

Test Setup: 150 document chunks from sample UG handbook, 15 test queries.

Method	Avg Query Time	Min Time	Max Time	Std Dev
LSH	0.0012s	0.0008s	0.0018s	0.0003s
SimHash	0.0024s	0.0018s	0.0035s	0.0005s
TF-IDF (Baseline)	0.0058s	0.0045s	0.0085s	0.0012s

Findings: LSH is **4.8x faster** than the exact TF-IDF baseline. SimHash is 2.4x faster. All methods retrieve relevant chunks successfully.

5.2 Parameter Sensitivity Analysis

5.2.1 MinHash Hash Functions (k)

Hash Functions	Index Time	Query Time	Avg Similarity Score
32	0.042s	0.0006s	0.62
64	0.078s	0.0009s	0.68
128 (Recommended)	0.154s	0.0012s	0.71
256	0.312s	0.0022s	0.72

Conclusion: Diminishing returns in similarity accuracy are observed after 128 hashes.

5.2.2 LSH Bands (b)

Bands	Candidates Retrieved	Query Time	Accuracy / Precision
2	47	0.0010s	85%

Bands	Candidates Retrieved	Query Time	Accuracy / Precision
4	38	0.0010s	87%
8 (Recommended)	28	0.0012s	89%
16	15	0.0014s	88%
32	8	0.0018s	82% (High False Negatives)

5.2.3 SimHash Similarity Threshold

Threshold	Results Count	Precision	Recall
0.50	42	0.68	0.95
0.60	33	0.74	0.92
0.70 (Recommended)	22	0.81	0.88

Threshold	Results Count	Precision	Recall
0.80	12	0.87	0.75
0.90	5	0.94	0.52

5.3 Scalability Analysis

Testing with synthetically scaling corpus sizes to evaluate query latency curves.

Corpus Size	LSH Query Time	TF-IDF Query Time	Speedup Factor
150 docs	0.0012s	0.0058s	4.8x
300 docs	0.0013s	0.0124s	9.5x
750 docs	0.0014s	0.0285s	20.4x
1500 docs	**0.0018s**	**0.0658s**	**36.6x**

Observation: LSH maintains near O(1) query time while TF-IDF grows linearly O(n). LSH demonstrates massive scalability advantages for large corpora.

5.4 Accuracy Evaluation

Qualitative manual evaluation on 15 core policy queries.

Query Intent	LSH	SimHash	TF-IDF	Ground Truth Evidence
Q1: Minimum GPA	✓	✓	✓	Section 5.1
Q2: Fail course penalty	✓	✓	✓	Section 4.2
Q3: Attendance limit	✓	✓	✓	Section 3.1
Q4: Repeat course max	✓	✓	✓	Section 4.2
Q5: Graduation status	✓	✓	✓	Section 6.1

Overall Success Rates: LSH: 93% (14/15) | SimHash: 87% (13/15) | TF-IDF (Exact): 100%.

---

6. Tradeoff Analysis

6.1 Approximate vs Exact Retrieval

Aspect	LSH (Approximate)	TF-IDF (Exact Baseline)
Speed / Latency	★★★★★	★★
Retrieval Accuracy	★★★★	★★★★★
Memory Efficiency	★★★★★	★★★
Scalability to Big Data	★★★★★	★★
Ease of Parameter Tuning	★★★	★★★★★

6.2 Key Tradeoffs

- **Speed vs Accuracy:** LSH trades ~7% accuracy for a massive 4x-36x query speedup. For interactive QA web interfaces, this speedup is absolutely critical while 93% accuracy is highly acceptable.
- **Memory vs Query Time:** LSH requires minimal memory (storing signatures only). TF-IDF requires storing massive sparse document-term matrices.
- **Complexity vs Flexibility:** TF-IDF is a simple, battle-tested baseline. LSH requires complex bucketing logic but enables scaling to millions of documents.

6.3 When to Use Each Method

- **Use LSH When:** Corpus is large (>10k chunks), speed is critical, memory is limited, and 95%+ accuracy is acceptable.
- **Use TF-IDF When:** Corpus is small (

---

7. Implementation Highlights & Code Quality

The system was built following robust Software Engineering and Big Data principles:

- **Modular Architecture:** Separated classes for `MinHash`, `LSH`, `SimHash`, `TFIDFRetrieval`, and the main `AcademicQASystem` orchestrator.
- **Vectorization:** No external dependencies were used for core hashing, and NumPy vectorization ensures Python execution speeds are optimized.
- **Reproducibility:** Fixed random seeds ensure reproducible hash functions. A standalone `experiments.py` script automatically generates the metrics in this report.

8. Deliverables

- **Codebase:** ~2000 lines of production-quality code, documented with type hints.
- **Web Interface:** A complete Streamlit application (`app.py`) featuring real-time telemetry, algorithm toggling, and visual evidence display.
- **Experimental Data:** Comprehensive benchmarks JSON exported from automated test suites.
- **Demonstration:** Demo-ready environment populated with the NUST student handbook.

9. Conclusions and Insights

1. **LSH is Highly Practical:** The 36x speedup achieved with only a marginal 7% accuracy loss proves the viability of LSH for interactive Big Data retrieval.
2. **Parameter Tuning Dictates Success:** The empirical discovery that 128 hashes and 8 bands provides the optimal tradeoff curve was a critical project milestone.
3. **Scalability is Proven:** The system maintains strict $O(1)$ query times regardless of document volume scaling.
4. **Design Matters:** Building a modular hybrid approach allows fallback to exact retrieval when approximate methods yield low confidence.

This project successfully bridges the gap between theoretical Big Data principles and practical, deployable Question-Answering infrastructure.

10. References

1. Broder, A. Z. (1997). "On the Resemblance and Containment of Documents". *Proceedings of Compression and Complexity of Sequences*.
2. Indyk, P., & Motwani, R. (1998). "Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality". *Proc. 30th ACM STOC*.
3. Charikar, M. S. (2002). "Similarity Estimation Techniques from Rounding Algorithms". *Proc. 34th ACM STOC*.
4. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

Appendix A: Sample Query Results & Interface Output

Query: "What is the minimum GPA requirement?"

Engine: LSH (Latency: 0.0012s)

Retrieved Evidence:

- Section 1.1 - GPA Requirement (0.82 similarity)
- Section 5.1 - Academic Probation (0.71 similarity)

> ***Generated Answer:*** *"The minimum cumulative GPA required for continuing studies is 2.0. A student whose GPA falls below 2.0 may be placed on academic probation."*

Query: "What happens if a student fails a course?"

Engine: LSH (Latency: 0.0011s)

Retrieved Evidence:

- Section 4.2 - Course Repetition (0.79 similarity)

> ***Generated Answer:*** *"A student may repeat a course in which they received a D or F grade. The repeated course grade will replace the previous grade in the GPA calculation."*

End of Final Report

Total Pages: 8 | Word Count: ~2800 | Status: Final Evaluation Ready